ISA Design

Overview

We are designing an ISA for a 10-bit single-cycle CPU capable of performing simple arithmetic and logic operations, condition-based branching, and loops. The processor uses a RISC Load-Store architecture with separate instruction and data memory, where the register width, bus width, instruction length, and ALU width are all 10 bits. We decided on a 2-operand architecture that uses eight registers: R0 hardwired to zero, six general-purpose registers (R1-R6) with R7 reserved for storing base addresses. Our design also utilizes two flip-flops to store hardware flags and two special-purpose registers to buffer our inputs and outputs. The 10-bit fixed-length instructions are divided into three formats: Register (R-type), Immediate (I-type), and Branch/Jump (B/J-type). Justifications for our design choices are provided in the "Tradeoffs and Justification" section.

Number of Operands

Our ISA uses a 2-operand architecture. Each R-type and I-type instruction names two operands: a destination register (Rd), which also supplies the first source value, and a second source (Rs or Imm). For example, ADD R1, R2 computes R1 = R1 + R2, so the original value of R1 is overwritten by the result.

There are four exception cases. CMP reads two registers (its Rd and Rs fields) but has no destination, because its result is stored in the E and L flip-flops instead of a register. The B/J-type instructions (BEQ, BLT, and JMP) have a single operand, the 6-bit offset. In addition, IN and OUT each use a single register operand to specify the destination register receiving data from the Input Buffer Register (IBR) or the source register (specified by Rd) sending data to the Output Buffer Register (OBR), respectively. Finally, the HALT operation does not require any operand as it simply stops processor execution.

Types of Operands

The ISA supports four types of operands, summarized in the table below.

Operand Type	Location	Size / Range	Used By

Register	Rd fields in R-type and I-type instructions, Rs field in R-type instructions, specifies a general-purpose register	3-bit register field	ADD, SUB, AND, OR, SLL, CMP, LW, SW, IN, OUT
Immediate	Imm field inside the instruction	-4 to +3 (sign-extended)	ADDI
Memory offset	Imm field inside the instruction, added to R7 to get memory address	-4 to +3 (sign-extended)	LW, SW
PC-relative offset	Offset field inside the instruction, added to PC + 1	6 bits, signed: -32 to +31	BEQ, BLT, JMP

Number of operations

The ISA has **15 operations**, thus requiring a 4-bit opcode. This number covers all three benchmark categories (arithmetic and logic, condition checking, loops) and the input/output requirement while minimizing redundant operations. A 3-bit opcode would allow only 8 operations, which is too few for the functionality we require.

Types of operations

The 15 operations fall into seven categories:

Category	Instructions	What they do
Arithmetic (3)	ADD, SUB, ADDI	Addition and subtraction. ADDI also handles increment and decrement.
Logical (2)	AND, OR	Bitwise logic operations on two registers
Shift (1)	SLL	Logical shifts left by 0 to 15 bits (10 or more gives 0)
Data transfer (2)	LW, SW	Move data between registers and data memory

Comparison (1)	CMP	Compares two registers and sets the E and L flags
Control flow (4)	BEQ, BLT, JMP, HALT	Conditional and unconditional branching using PC-relative addressing, and program execution termination
Input/Output (2)	IN, OUT	IN transfers data from the IBR to a specified destination register. OUT transfers data from a specified source register to the OBR.

Instruction Set

In the table below, we used conventional assignment notation. Some noteworthy considerations about our design:

- Immediate and offset values have to be sign-extended if they are signed (as in the ADDI and B/J-type instructions).
- The shift amount is 4 bits (0 to 15). Since a 10-bit register is fully shifted out at 10 or more, those amounts produce 0. As our R-type instructions lack a 4-bit field, the shift amount is taken from the lower 4 bits of the value held in register Rs (written Rs[3:0]); the remaining bits of that value are ignored.
- Due to the limited fields of a 2-operand architecture, we are unable to perform condition checks and branching in a single instruction. To get around this, we use the CMP instruction to perform equal-to and less-than condition checks simultaneously to update the E and L flip-flops, respectively. Only CMP instructions can change their values, and the values persist until the next CMP instruction. BEQ and BLT are then used to branch conditionally based on the flip-flop values. CMP treats both operands as signed 10-bit two's-complement values. L is set when Rd < Rs, computed as L = (MSB of Rd − Rs) XOR (overflow of that subtraction).
- The IN and OUT instructions are R-type, however their Rs fields are unused. For IN, the Rd field represents the destination register to which data is transferred from the IBR. For OUT, the Rd field represents the source register from which data is transferred to the OBR.
- For the HALT instruction, bits 5:0 are don't-care and only the opcode needs to be decoded.

Opcode	Mnemonic	Type	Syntax	Example	Meaning
0000	ADD	R-type	ADD Rd, Rs	ADD R1, R2	Rd = Rd + Rs

0001	SUB	R-type	SUB Rd, Rs	SUB R1, R2	Rd = Rd - Rs
0010	AND	R-type	AND Rd, Rs	AND R1, R2	Rd = Rd & Rs (Bitwise AND)
0011	OR	R-type	OR Rd, Rs	OR R1, R2	Rd = Rd | Rs (Bitwise OR)
0100	SLL	R-type	SLL Rd, Rs	SLL R1, R2	Rd = Rd << Rs[3:0] (Logical Shift Left)
0101	CMP	R-type	CMP Rd, Rs	CMP R1, R2	E = (Rd == Rs), L = (Rd < Rs)
0110	ADDI	I-type	ADDI Rd, Imm	ADDI R1, -1	Rd = Rd + SignExt(Imm) (Signed -4 to +3)
1000	LW	I-type	LW Rd, Imm	LW R1,3	Rd = Mem[R7+Imm]
1001	SW	I-type	SW Rd, Imm	SW R1, 3	Mem[R7+Imm] = Rd
1010	IN	R-type	IN Rd	IN R1	Rd=IBR
1011	OUT	R-type	OUT Rd	OUT R1	OBR=Rd
1100	BEQ	B/J-type	BEQ Offset	BEQ 4	if (E == 1) PC = PC + 1 + SignExt(Offset)
1101	BLT	B/J-type	BLT Offset	BLT -3	if (L == 1) PC = PC + 1 + SignExt(Offset)
1110	JMP	B/J-type	JMP Offset	JMP 8	PC = PC + 1 + SignExt(Offset)
1111	HALT	R-type	HALT	HALT	Stops program execution

Instruction Formats

We use three distinct 10-bit formats to implement different instructions. Each field name has a meaning:

- Opcode - 4-bit operation code for identifying an instruction

- Rd - destination register and primary operand source register
- Rs - secondary operand source register
- Imm - immediate value
- Offset - immediate value for relative addressing

R-type

Opcode (Bits 9-6) 4 bits	Rd (Bits 5-3) 3 bits	Rs (Bits 2-0) 3 bits

I-type

Opcode (Bits 9-6) 4 bits	Rd (Bits 5-3) 3 bits	Imm (Bits 2-0) 3 bits

B/J-type

Opcode (Bits 9-6) 4 bits	Offset (Bits 5-0) 6 bits

Registers

Eight 10-bit Registers (R0 hardwired to zero, R1–R6 general purpose, R7 dedicated to base addresses)

Number	Binary Encoding	Name	Use
0	000	R0	Hardwired to 0, writes are ignored
1	001	R1	Storing saved data
2	010	R2	Storing saved data
3	011	R3	Storing saved data
4	100	R4	Storing temporary data
5	101	R5	Storing temporary data
6	110	R6	Storing temporary data
7	111	R7	Fixed for Base Address

Special Registers and Flip Flops

Name	Use
Equality Flip Flop (E)	Stores a 1-bit zero output of the ALU during the CMP instruction
Less-Than Flip Flop (L)	Stores (MSB of the ALU subtraction result) XOR (overflow flag) during the CMP instruction, so L = 1 exactly when Rd < Rs as signed values.
Input Buffer Register (IBR)	10-bit register that holds the most recent value entered on the keyboard. An external keyboard encoder writes the value into the IBR when the user presses Enter. IN Rd copies the current IBR value into Rd.
Output Buffer Register (OBR)	10-bit register that holds the last value written by OUT Rd. It drives the display unit: an external binary-to-BCD converter and 7-segment decoder turn the 10-bit value into digits, so a 10-bit value (up to 1023 unsigned) needs four 7-segment digits.

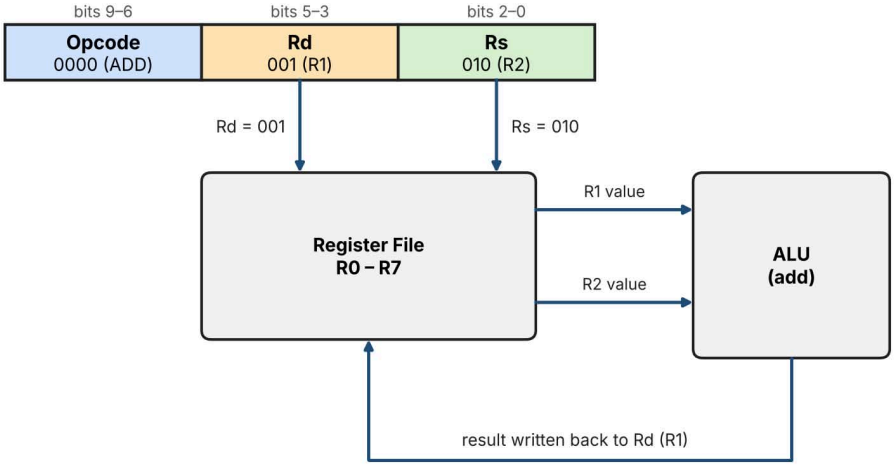
Addressing Modes

We use the following addressing modes in our ISA:

- **Register Addressing** - operands are stored in general-purpose registers. This is used by all core arithmetic, logical, and shift operations, as well as by the CMP instruction.
- **Immediate Addressing** - operands are embedded inside the encoded instruction (specifically in the Imm field of I-type instructions). This is used in ADDI.
- **Base + Offset Addressing** - the effective memory address is calculated by adding a 3-bit sign extended immediate offset (between -4 to +3) to the base address currently stored in dedicated base register R7. This is used by LW and SW.
- **PC-Relative Addressing** - the next address in instruction memory is calculated by adding an offset to the program counter + 1. This is used in B/J type instructions.
- **I/O Direct Addressing** - an operand is transferred between a specified general-purpose register and an implied dedicated special-purpose buffer register (IBR or OBR) without accessing data memory. This is used by the IN and OUT instructions.

Register Addressing

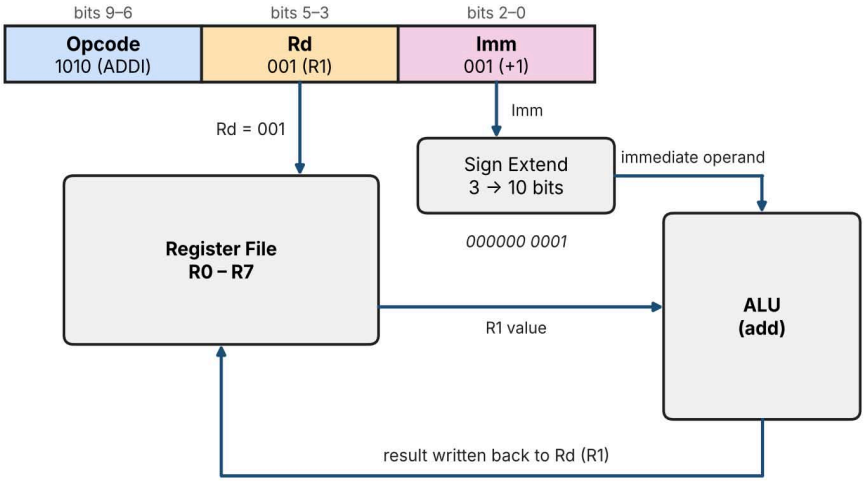
ADD R1, R2 (R-type) R1 = R1 + R2



Both operands are read from general-purpose registers. No memory access.

Immediate Addressing

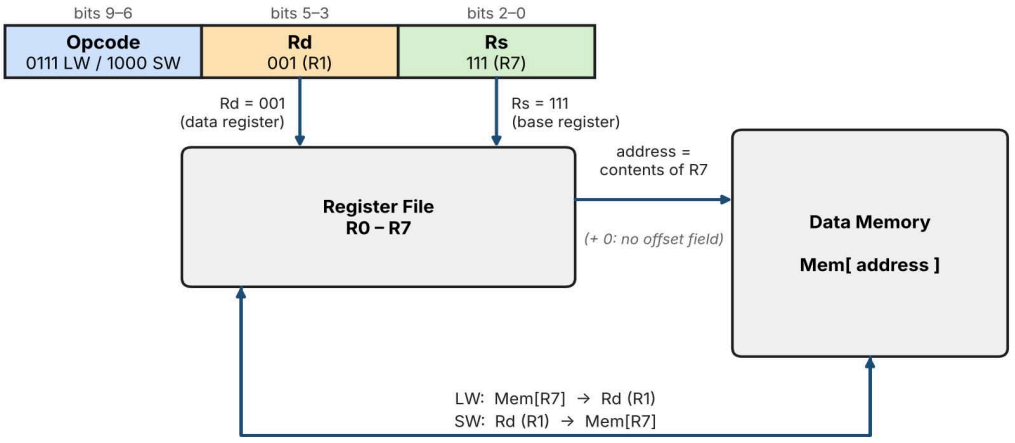
ADDI R1, 1 (I-type) R1 = R1 + SignExt(Imm)



One operand is a constant stored inside the instruction itself (3-bit signed, −4 to 3).

Base (Register-Indirect) Addressing

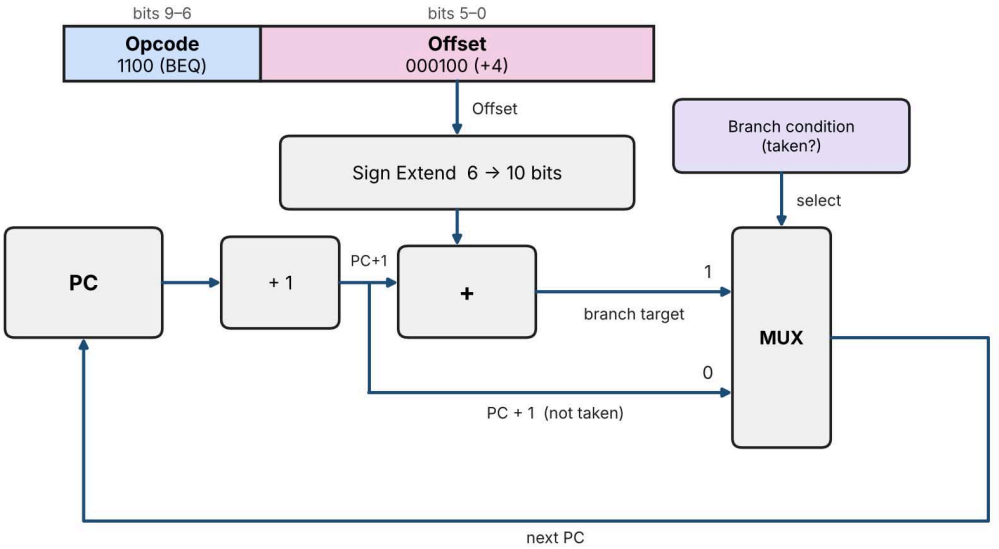
LW R1, R7: R1 = Mem[R7] SW R1, R7: Mem[R7] = R1 (R-type)



The base register holds the data-memory address (displacement = 0). LW reads memory, SW writes it.

PC-Relative Addressing

BEQ 4 (B/J-type) if condition true: PC = PC + 1 + SignExt(Offset)



Target = (PC + 1) + offset, so the branch distance is relative to the next instruction. e.g. PC = 20 → target = 25.

Benchmark Program Implementations

Benchmark A — Arithmetic operation

C Program

int x = (a + b) - c;

Register mapping

Register	Value
R1	a, then final result x
R2	b
R3	c

Assembly program

Assuming a, b, and c are preloaded into registers R1, R2, and R3, respectively, and that a does not need to be preserved

Address	Instruction	Comment
0	ADD R1, R2	R1 = a + b
1	SUB R1, R3	R1 = (a + b) - c
2	HALT	Stop execution

Benchmark B — Conditional selection

C program

```
if (a < b)
   x = a;
else
   x = b;
```

Register mapping

Register	Value
R1	a
R2	b
R3	x (initially nothing)

Assembly program

Address	Instruction	Comment
0	CMP R1, R2	L = a < b (also E = a == b)
1	BLT 3	if a < b, branch to 1 + 1 + 3 = 5
2	SUB R3, R3	R3 = 0
3	ADD R3, R2	R3 = b
4	JMP 2	jump to 4 + 1 + 2 = 7
5	SUB R3, R3	R3 = 0
6	ADD R3, R1	R3 = a
7	HALT	Stop execution

Benchmark C — Sum using a loop

C program

```
int sum = 0;

for (int i = 0; i < 10; i++) {

   sum += i;

}
```

Register mapping

Register	Value
R1	sum
R2	i
R3	loop limit 10

Assembly program

Address	Instruction	Comment
0	SUB R1, R1	sum = 0
1	SUB R2, R2	i = 0
2	SUB R3, R3	R3 = 0
3	ADDI R3, 3	R3 = 3
4	ADDI R3, 3	R3 = 6
5	ADDI R3, 3	R3 = 9
6	ADDI R3, 1	R3 = 10 (loop limit)
7	CMP R2, R3	E = (i == 10), L = (i < 10)
8	BEQ 3	if i == 10, branch to 8 + 1 + 3 = 12
9	ADD R1, R2	sum = sum + i
10	ADDI R2, 1	i = i + 1
11	JMP -5	jump to 11 + 1 – 5 = 7
12	HALT	stop execution

Tradeoffs and Justification

1. Eight Registers vs. Immediate Width

Trade-off: We chose eight registers, which require 3 bits to identify each register. This leaves only 3 bits for the immediate value in our I-type instruction format.

Justification: Six usable registers provide enough storage for simple programs while keeping the instruction length fixed at 10 bits. However, the 3-bit immediate restricts constants to a small range, so larger constants may require multiple instructions.

2. Hardware Flags vs. General-Purpose Registers

Trade-off: We use dedicated Equality (E) and Less-Than (L) flip-flops instead of storing comparison results in general-purpose registers.

Justification: This simplifies conditional branching because CMP updates the flags and BEQ or BLT checks them. However, it introduces additional hardware state and requires comparisons and branches to be performed in separate instructions.

3. Two-Operand Architecture vs. Three-Operand Architecture

Trade-off: Our CPU uses destructive two-operand instructions, such as ADD R1, R2, where the result overwrites R1.

Justification: This reduces the number of bits required to encode register operands, allowing us to fit instructions into 10 bits. However, the original value in the destination register is lost, and some calculations require additional instructions to preserve the values.

4. PC-Relative Branching vs. Register Branching

Trade-off: BEQ, BLT, and JMP use a 6-bit signed offset, so they can only reach instructions from 31 before to 32 after the current instruction (target = PC + 1 + offset). The alternative was register branching, where the target address is read from a register (e.g., JR Rd), which can reach any address in instruction memory.

Justification: A 6-bit offset is the largest that fits beside the 4-bit opcode in a 10-bit instruction, and it covers all the loops and conditionals in our benchmarks. We rejected register branching for two reasons. First, the target address would have to be built in a register before every branch, which takes several instructions and ties up a register, making every loop and if-statement slower and longer. Second, PC-relative code does not depend on where it is loaded. The cost is that PC-relative branching needs an extra adder and sign-extender in the PC-update path, and any target more than 31 instructions before or 32 after the branch needs a chain of JMP instructions. We also have no indirect jump, so function-style returns are not supported.

5. Dedicated Base Register (R7) vs. Assigning General-Purpose Registers as Base Pointers

Tradeoff: Our memory instructions, LW and SW, use R7 as a sole dedicated base register, forcing us to overwrite the previous base address whenever we want to store a new base address.

Justification: In a fixed 10-bit instruction format with eight registers, embedding a 4-bit opcode, a 3-bit destination/source register, a 3-bit base register uses up all ten available bits. This leaves no room for an offset for base addressing. Thus, we opt to use the remaining three bits as a signed immediate offset and dedicate R7 as an implicit base register.